# Adaptive Robot Planning

Master's Internship Report

Internship carried out at **Vrije Universiteit Amsterdam (Netherlands)**
as part of the **Master in IRobot– University of Montpellier**


## Author:

Luca

## Supervisors:

Dr. Filip Ilievski (Vrije Universiteit Amsterdam)

Prof. Madalina CROITORU (University of Montpellier)

## Contributors / Discussion Group:

Xinrui Zu
Stefano De Giorgis
Mohammadhossein Khojasteh


**Period:** June–July 2025
**Academic Year**: 2024–2025

**Adaptive Robot Planning** **1**
**Master's Internship Report** **1**
**1 Introduction** **4**
**2 Related Work** **5**
2.1 Assistive Robotics for Physical Guidance 5
2.2 Commonsense Reasoning and Knowledge Graphs 5
2.3 Multimodal Interaction in Human–Robot Interaction 5
2.4 LLM-based Planning and Reasoning 6
**3 System Architecture** **7**
3.1 Perception 7
(a) Object detection 7
(c) Task monitoring 8
(d) Speech-to-text (STT) 8
3.2 Reasoning 9
(a) Context integration 9
(b) LLM-based adaptive reasoning 10
(c) Response verification 10
(d) Output to execution 10
3.3 Action Execution 11
(a) Command interpretation 11
(b) Physical execution of movements 12
(c) Verbal feedback and synchronization 13
(d) Safety supervision and monitoring 13
3.4 Planning and Adaptation 14
(a) Global routine structure 14
(b) Local adaptations 14
(c) Balancing scripted and adaptive behaviour 14
**4 Implementation** **15**
4.1 Software Stack 15
4.2 Prompt Engineering and Response Format 16
4.3 Knowledge Representation 17
4.4 Hardware Setup 17
**5 User Evaluation Design** **18**
**6. Results and Discussion** **20**
6.1 Quantitative results 20
6.2 Qualitative feedback 21
6.3 Technical and interaction-level observations 21
6.4 Interpretation and hypotheses 22
6.5 Strengths and limitations 22
**7. Conclusion and Perspectives** **22**
7.1 Summary of findings 22
7.2 Implications for assistive robotics 22
7.3 Generalizability to other use cases 23
7.4 Future work 23

# 1 Introduction

Robotic systems are increasingly deployed in domestic and healthcare environments to support human wellbeing through physical assistance, rehabilitation, and guided exercise routines [1,2]. Among these, assistive stretching robots have shown potential for improving flexibility, reducing stress, and encouraging physical activity in daily life [3]. However, most existing systems remain scripted and inflexible, providing limited personalization to the user's physical state, emotional condition, or surrounding context [4]. This lack of adaptation can reduce engagement, hinder safety, and limit long-term adoption.

To address these challenges, this work investigates whether and how adaptive robotic guidance can outperform scripted routines in terms of personalization, naturalness, and contextual relevance. In particular, we examine to what extent multimodal perception combined with commonsense reasoning enables a robot to adapt its guidance to the user's state and environment. Building on this research question, we developed **StretchBot**, an adaptive robotic coach designed to guide users through a short morning stretching routine while dynamically adjusting its behavior to real-time multimodal cues. StretchBot integrates perception, commonsense reasoning, and interaction control in a unified architecture, enabling it to perceive user state through voice, facial expression, and posture, detect relevant objects in the environment, and adapt stretching plans accordingly. The system is connected to a robotic arm, which can point to perceived objects that may assist the user during the routine.

At the core of our system lies a hybrid reasoning module that couples a knowledge graph encoding causal, temporal, and affordance-based relations with a large language model (LLM) for context-aware decision-making [5]. This reasoning pipeline is augmented with a verification loop using a secondary language model to ensure safety, protocol adherence, and empathetic communication before actions are executed. The result is a robot capable of proactive yet collaborative guidance.

We validate StretchBot through a comparative user study contrasting a scripted baseline with the adaptive system. Three participants tested each version, providing ratings on clarity, comfort, adaptability, trust, naturalness, and object relevance. Results indicate that while the adaptive robot was perceived as significantly better at adapting to the user and pointing to relevant objects, some participants overall preferred the scripted robot because of its predictability and smoother flow. Comfort and trust were rated similarly between the two versions.

The main contributions of this work are:

- An adaptive stretching robot framework integrating multimodal perception, commonsense reasoning, and interaction control for guided physical routines.

- Integration of knowledge graph reasoning with large language models prompt design, enabling the robot to adapt exercise plans based on user state, environment, and prior interaction history.

- A collaborative interaction protocol ensuring explicit user confirmation before all actions, balancing proactive adaptation with human oversight.

- Comparative human evaluation of scripted versus adaptive guidance in a real-world stretching scenario.

# 2 Related Work

## 2.1 Assistive Robotics for Physical Guidance

Robots have been explored as assistants for stretching and rehabilitation exercises, aiming to improve user wellbeing and engagement. Previous work [6] showed that scripted stretching robots can help users maintain focus and reduce stress during routines. However, such systems are limited by their lack of real-time adaptation to user state. Another study [7] proposed a rehabilitation robot for Parkinson's patients that adapted exercises over time, but the adaptation relied on fixed predefined rules. In contrast, StretchBot combines multimodal perception with commonsense reasoning to adapt instructions continuously during a session, allowing for more personalized and context-aware guidance.

## 2.2 Commonsense Reasoning and Knowledge Graphs

*Commonsense reasoning has been used to give robots a better understanding of their environment and the needs of the user. Prior work [8] proposed 13 dimensions of commonsense knowledge, which inspired the organisation of our knowledge graph. Another study [9] showed that combining symbolic planning with language models can fill gaps in action reasoning. In our case, commonsense reasoning is applied to connect perceived user states, such as fatigue or thirst, with relevant object affordances, allowing the robot to decide when to suggest a break or indicate a helpful object.*

## 2.3 Multimodal Interaction in Human–Robot Interaction

*Prior work has shown that combining multiple sensing modalities improves the adaptability and perceived quality of human–robot interaction. For example, researchers [10] used emotion and posture detection to adjust guidance, while other work [11] adapted robot behaviour in response to signs of fatigue. StretchBot integrates emotion, posture, and*

*environmental object cues into a reasoning process that supports timely and context-aware assistance, such as proposing a pause or pointing to a relevant object.*

## 2.4 LLM-based Planning and Reasoning

*Recent work has explored the use of large language models for robotic planning. One approach [12] introduced SayCan, combining LLM-based reasoning with affordance detection to generate feasible action plans. Another study [13] demonstrated that maintaining an "inner monologue" allows LLMs to revise plans in real time, increasing success rates. Several surveys [14] have identified strengths and weaknesses of LLM reasoning, while recent contributions at major conferences such as ICML [14] have highlighted the need for hybrid approaches. Other work [15] further explored how structured knowledge can improve manipulation planning. StretchBot applies these insights to the domain of physical guidance, coupling LLMs with a structured knowledge graph and a verification loop to ensure safe, empathetic, and contextually appropriate robot actions*

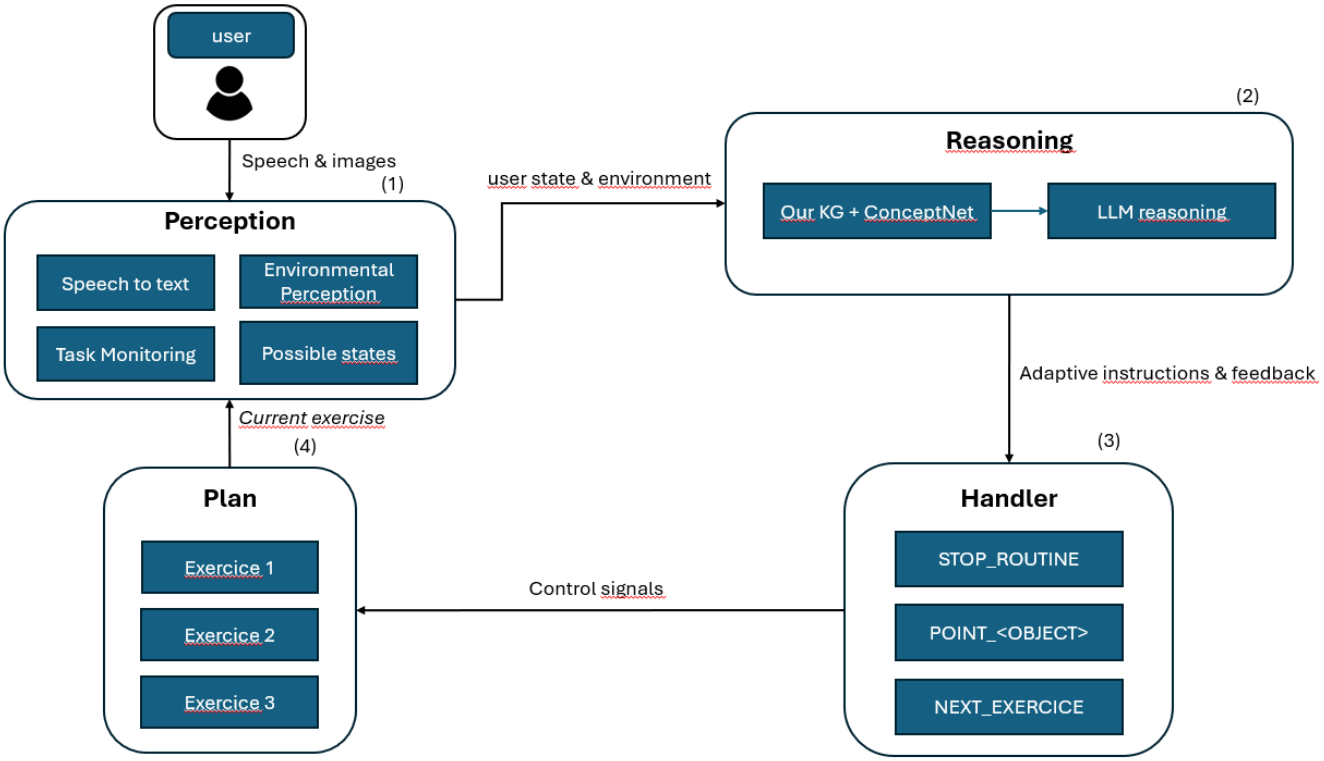


Figure 1: overall pipeline. The framework is organized into four main stages: Perception, Reasoning, Action Execution, and Planning & Adaptation. Information flows clockwise, from perception of the user and environment to reasoning, planning, and finally execution, forming a closed loop that allows the robot to adapt its behavior in real time.

# 3 System Architecture

StretchBot is structured around this four-stage pipeline (Figure 1). Each stage transforms the information it receives into increasingly higher-level representations, enabling the robot to adapt its behavior in real time. The Perception module collects multimodal data from the user and the environment. The Reasoning module interprets this data using a combination of knowledge graphs and large language models. Planning organizes the stretching routine based on the interpreted context, while Action Execution carries out physical movements and provides verbal feedback. This modular design allows the system to be both extensible and robust, supporting adaptive behaviors without sacrificing safety or clarity [16,17].

## 3.1 Perception

The perception module provides the adaptive system with a continuous and multimodal understanding of both the user's state and the environment. It integrates four complementary components: object detection, multimodal emotion recognition, task monitoring, and speech-to-text (STT). Together, they supply structured information to the reasoning module for context-aware decision-making.

### (a) Object detection

Visual scene analysis is performed using YOLOv8 [18], applied to the live RGB feed from the robot's camera. The system retrieves the complete list of all detected objects in the scene, and sends this list to the reasoning module. This ensures that every recognized element, whether it is directly relevant to the current exercise, is available for potential reasoning or adaptation.

**(b) Multimodal emotion recognition**

The user's emotional state is inferred through the fusion of three independent detection channels:

- **Voice emotion recognition** – using a HuggingFace Transformers pipeline [19] on speech segments captured by the microphone.
- **Facial emotion recognition** – using DeepFace [20] applied to the video stream.
- **Text-based emotion recognition** – using a lightweight HuggingFace model [19] on transcribed speech.

Each channel outputs an emotion label (e.g., happy, neutral, angry) with an associated confidence score. Relying on a single modality would be fragile, as speech may be affected by background noise, facial expressions may be ambiguous or occluded, and text transcriptions may lack emotional nuance. By combining the three channels, the system increases robustness and reliability, ensuring that temporary errors in one modality are compensated by the others.

A dedicated fusion module performs weighted fusion and majority voting to produce a single aggregated emotional state, which is then summarized in natural language for the LLM.

### (c) Task monitoring

Task monitoring ensures that the user is following the current stretching exercise correctly. It uses posture estimation methods [21] to verify whether the expected pose is being held, and simultaneously measures the hold duration. When the predefined time threshold has elapsed, the exercise is considered to be successfully completed. This information is then communicated to the reasoning module, where it is interpreted by the LLM as a signal to proceed with the next step of the routine.

### (d) Speech-to-text (STT)

The STT component converts the user's spoken input into text for downstream processing, enabling both textual emotion recognition and natural-language interaction with the LLM. For this, Vosk [22] was initially used due to its lightweight local deployment, with Google Cloud Speech-to-Text considered for higher accuracy [23].

All perception outputs are structured into a unified context package containing the detected objects, the aggregated emotional state, the current task execution status, and the most recent STT transcript. This package is transmitted to the reasoning module whenever the user poses a new question, ensuring that action planning remains aligned with the user's state and the environment in real time.

**Example (running scenario):** Consider the case where the user says *"I didn't sleep well last night"* while attempting an arm stretch. The perception module processes this situation through four components:

1. **Object detection** identifies a coffee mug, a banana, and a glass of water in the environment → *output: [Coffee, Banana, GlassOfWater]*.

2. **Multimodal emotion recognition** classifies the user as tired based on voice tone and facial expression → *output: tired*.

3. **Task monitoring** verifies that the arm stretch has started but not yet completed → *output: exercise = arm stretch, status = in progress*.

4. **Speech-to-text** transcribes the utterance → *output: "I didn't sleep well last night."*

Together, these outputs form the unified perception context sent to the reasoning module.

## 3.2 Reasoning

The reasoning module serves as the central decision-making component of the system, responsible for determining the robot's next action by combining perception outputs, prior knowledge, and the predefined stretching routine. Its operation follows a hybrid approach: deterministic, rule-based checks handle conditions that can be explicitly defined, such as verifying when an exercise's hold duration has been reached, while a Large Language Model (LLM) provides high-level reasoning, contextual adaptation, and natural-language interaction [24], [25].

### (a) Context integration

Each time the user speaks, the reasoning module compiles an updated context package. This package includes:

– the list of objects detected by the YOLO-based perception system,
– the aggregated emotional state derived from multimodal emotion recognition (voice, face, text),
– the current progress status of the ongoing exercise provided by task monitoring,
– and the latest speech-to-text (STT) transcription of the user's utterance.

To provide the LLM with meaningful background knowledge, this raw context is enriched with additional semantic relations retrieved from a Knowledge Graph (KG). A knowledge graph is a structured representation of concepts (nodes) and their relationships (edges), often expressed as triples like *(coffee, reduces, fatigue)* [26]. Unlike unstructured text, a KG enables the system to reason over entities and their connections in a transparent and interpretable way.

When an entity detected by perception (e.g., coffee, banana) or an emotional state (e.g., anger, tired) appears in the context, the system queries the KG specifically for relations involving that entity. This targeted approach ensures that only relevant background knowledge is included. For instance, querying the KG with *anger* may return relations such as *anger can be reduced by sympathy*, while querying with *coffee* can yield *coffee reduces fatigue*. These relations are then injected into the context package, enabling the LLM to adapt its responses based on both perception and commonsense knowledge [5], [8].

If the queried entity is not found in the internal KG, the system falls back on ConceptNet, a large-scale, publicly available commonsense knowledge base [5]. ConceptNet connects everyday concepts through relations like ***IsA, Causes, or UsedFor***. For example, it encodes commonsense facts such as *stretching improves flexibility* or *drinking water reduces thirst*. Using ConceptNet as a fallback guarantees broader coverage of entities and situations, even when they are not present in the custom KG.

This mechanism avoids overloading the LLM with unnecessary relations by providing only those relevant to the user's current state and environment. In doing so, the reasoning

process remains both context-aware and efficient, ensuring that the robot's decisions are grounded in perception while enriched with structured commonsense knowledge.

## (b) LLM-based adaptive reasoning

The complete "context package" is passed to the primary LLM through a structured prompt. The model analyses this information to decide the most appropriate next step in the stretching session, adapting the style and tone of its response to match the detected emotional state of the user [12], [13], [14]. If relevant objects are available in the scene, the model may refer to them directly in its suggestions, such as recommending coffee when signs of fatigue are detected.

To ensure consistent communication between modules, the LLM adheres to predefined action prefixes in its outputs. For example, `NEXT_EXERCISE:` instructs the system to move on to the following stretch, `POINT_<OBJECT>:` directs the robot to point to a detected object, and `STOP_ROUTINE:` ends the stretching session. These prefixes are inserted directly before the natural-language portion of the message, so that the execution module can interpret the intended action while the user receives a friendly and engaging verbal response [27].

## (c) Response verification

Once the primary LLM has generated a response, it is passed to a secondary, smaller LLM acting as a verifier. This verification stage plays a crucial role in maintaining the quality and consistency of interactions. The verifier checks that the message follows the expected output format, confirms that the tone is appropriate for the user's emotional state, and ensures that the suggested action is coherent with the current session context. If the verification process detects an error, such as a missing action prefix or an inconsistent recommendation, it will adjust the message before it is sent to the execution layer [28].

## (d) Output to execution

After verification, the reasoning module delivers its response to the execution layer. In most cases, this consists of a natural-language sentence to be spoken by the robot. When required, the response also contains a structured action command for the control system (e.g., pointing to an object, moving to the next exercise, or stopping the routine). This design ensures that the robot's verbal and physical behaviours remain coordinated when both are present, while still allowing for purely conversational responses when no action is needed [29].

**Example (running scenario):** Based on the perception context (*objects = [Coffee, Banana, GlassOfWater]; emotion = tired; exercise = arm stretch, status = in progress; STT = "I didn't sleep well last night"*), the reasoning module queries the knowledge graph. The KG links *tired → energy boost → coffee or banana*. The LLM then generates a structured action: *POINT_Coffee*, accompanied by an empathetic utterance ("Oh, I see – mornings are tough

when you don’t sleep well. Maybe some coffee can help.”). A secondary verification model ensures the response is consistent with the user’s state, contains the correct action prefix, and respects the dialogue protocol before sending it to the execution module.

.

## 3.3 Action Execution

The action execution module is responsible for grounding high-level reasoning decisions in concrete robot behaviour. It acts as the interface between symbolic instructions produced by the reasoning module and the physical world, ensuring that every action is carried out in a synchronized, safe, and comprehensible manner for the user [30]. The execution process can be divided into four tightly coupled subcomponents: command interpretation, physical trajectory control, verbal feedback generation, and safety supervision.

### (a) Command interpretation

Each output from the reasoning stage arrives in a dual format: a structured action prefix and a natural-language sentence. The action prefix encodes the intended behaviour in a machine-readable form, while the accompanying text provides a human-friendly explanation or encouragement.

- `NEXT_EXERCISE` indicates progression in the stretching routine.
- `POINT_<OBJECT>` instructs the robotic arm to orient towards a detected object in the environment.
- `STOP_ROUTINE` signals the termination of the current session, ensuring that the system disengages gracefully.

This separation guarantees that verbal and physical layers remain synchronized while still allowing modular flexibility. The interpretation unit parses the prefix, triggers the corresponding execution pathway, and simultaneously queues the natural-language message for speech synthesis [27].

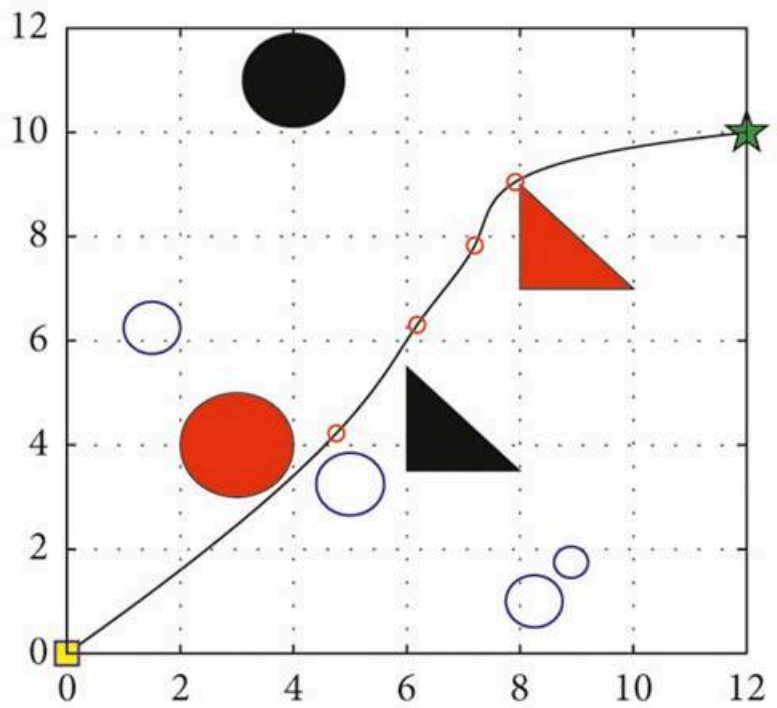


Figure 2: Example of a cubic spline interpolation ensuring smooth trajectory generation between discrete waypoints

## (b) Physical execution of movements

Once a symbolic command such as *POINT_Coffee* has been issued, the system translates it into a fixed Cartesian target pose defined by its position and orientation (x, y, z, roll, pitch, yaw). At this stage, the robot does not dynamically adapt to moving objects; instead, each recognized object is associated with predefined goal coordinates in the workspace.

The generated target is passed to MoveIt [31], which computes a feasible joint-space trajectory from the current configuration to the goal pose while avoiding collisions with the robot's kinematic constraints. The resulting trajectory is then optimized for real-time execution. The key step is the use of cubic spline interpolation [32], which ensures that the trajectory between waypoints remains smooth and continuous in both position and velocity (see Figure 2). This technique minimizes jerky movements by generating a globally consistent curve that passes through all planned waypoints.

The final optimized trajectory is stored as a sequence of joint positions, velocities, and timestamps. It is then executed step by step on the Lynxmotion SES900 arm [33]. Execution proceeds until the target pose is reached within tolerance, at which point an "execution finished" event is published back to the reasoning module. Once this signal is received, the arm maintains its pointing gesture for 5 seconds, after which it automatically returns to a predefined home position to reset its configuration for the next command.

### (c) Verbal feedback and synchronization

In parallel with motion control, the execution module delivers the verbal component of the response. Text-to-speech (TTS) converts the LLM's natural-language output into spoken instructions. The timing of speech and movement is carefully coordinated: for example, the pointing gesture is executed while the robot simultaneously explains the relevance of the object ("Here is a glass of water, in case you feel thirsty"). This multimodal synchronization enhances clarity and naturalness of the interaction [34].

The system supports multiple voices, ranging from simple rule-based synthesis to expressive neural voices capable of conveying emotion [35], reinforcing empathy and engagement.

### (d) Safety supervision and monitoring

A critical aspect of execution is maintaining user safety and hardware integrity. The action execution module integrates a watchdog subsystem that supervises all ongoing actions.

- **Joint limits and velocity clamping:** Every command is checked against predefined limits to prevent excessive extension or unsafe speed [36].
- **Collision detection:** The robot monitors its environment for potential collisions, either through geometric constraints or optional external tracking systems (e.g., OptiTrack) [37].
- **Emergency stop:** If anomalies are detected—such as unexpected resistance, trajectory divergence, or hardware malfunction—the system immediately halts execution and enters a recovery mode [38].

In summary, the action execution module anchors the adaptive reasoning of StretchBot in safe, synchronized, and interpretable behaviour. It not only ensures that verbal and physical channels remain aligned but also closes the loop by reporting execution outcomes back to the reasoning stage. This bidirectional flow allows the system to sustain an adaptive cycle where perception informs reasoning, reasoning directs execution, and execution feeds back into the next round of perception and decision-making.

**Example (running scenario):** Upon receiving the instruction *POINT_Coffee*, the execution module parses the prefix and triggers the corresponding motion. MoveIt computes a trajectory toward the mug's Cartesian coordinates, and the Lynxmotion SES900 arm executes the pointing gesture. Simultaneously, the text-to-speech module delivers the empathetic message: *"Here is some coffee, in case you need an energy boost."* The gesture and speech are synchronized, reinforcing the clarity and naturalness of the interaction. Safety checks (joint limits, collision detection, watchdog monitoring) run in parallel to ensure reliable execution.

## 3.4 Planning and Adaptation

The planning and adaptation module ensures that StretchBot delivers a stretching session that is both structured and responsive. It connects the high-level exercise routine with the contextual signals provided by perception and reasoning, allowing the robot to adapt in real time while avoiding loss of coherence [39], [40].

## (a) Global routine structure

At its core, the system relies on a predefined exercise script, composed of a sequence of stretching primitives (e.g., arm stretch, side bend, neck rotation). Each primitive specifies:

- the expected body posture,
- the target hold duration (5 s),
- and the transition to the next exercise.

This script acts as a baseline plan: it guarantees coverage of different body areas, prevents unnecessary repetitions, and maintains a clear beginning–middle–end flow. In other words, it anchors the session in a predictable backbone that can be adapted but never entirely discarded [41].

## (b) Local adaptations

On top of this baseline, the system introduces adaptive branching points, where the flow can be adjusted depending on the user's state. For example:

- If the perception module reports fatigue and an object such as coffee or water is detected, the reasoning module may insert a hydration break (`POINT_Water` + empathetic utterance).
- If the task monitoring detects that an exercise is not properly executed or the user reports discomfort ("my neck hurts"), the system can skip the current exercise and move to the next one [42].
- If the user interrupts the session ("I need to leave"), the reasoning outputs a `STOP_ROUTINE` command, ending the plan gracefully rather than abruptly [43].

## (c) Balancing scripted and adaptive behaviour

The challenge lies in combining predictability (so the user feels guided by a consistent program) with flexibility (so the interaction feels personalized). StretchBot achieves this by treating the routine as a **state machine**: the global plan defines the sequence of states (exercises), while adaptation allows conditional transitions between them [44], [45].

Importantly, the system never abandons the overall routine; even after a pause or a skipped step, it resumes from the most relevant next exercise.

This design ensures that the user perceives the robot as both trustworthy—because it follows a clear script—and attentive—because it reacts to physical and emotional cues in real time. The result is an interaction that feels simultaneously structured and personalized [46].

**Example (running scenario):** After the brief detour suggesting coffee, the planning module keeps the routine at the current exercise (arm stretch). The system waits for a new user input and a fresh reasoning cycle before any transition occurs. Only when the LLM explicitly outputs *NEXT_EXERCISE* does the robot advance to the next movement in the routine. In this way, local adaptations such as pointing to an object are integrated seamlessly without skipping exercises, and progression through the session always remains under the LLM's control.

# 4 Implementation

After presenting the overall architecture of StretchBot, we now describe how the system was put into practice. The implementation focuses on combining robotics frameworks with modern AI services to create a robot that can guide users through stretching routines in real time. The different modules introduced earlier — perception, reasoning, and interaction — were realized through a combination of open-source software, pretrained models, and lightweight hardware.

The design emphasizes modularity and accessibility: each component (speech recognition, emotion detection, object perception, reasoning, motion control) was implemented as an independent process but integrated within a unified pipeline. By relying on external inference for large models, the system remains affordable and can run on consumer hardware, while still enabling advanced reasoning and adaptive

## 4.1 Software Stack

StretchBot is built on a modular software stack centered around **ROS 2 Humble**, which manages communication between nodes and synchronizes with **MoveIt** for motion planning [31]. Physical simulation is handled in **PyBullet**, allowing trajectories to be tested before deployment on the real robot **Lynxmotion SES900** [33]. The entire system is developed in **Python 3.10** and integrates specialized models for perception, reasoning, and interaction.

Multimodal emotion recognition is achieved through three complementary channels: speech is analyzed with *ehcalabres/wav2vec2-lg-xlsr-en-speech-emotion-recognition*, text with *michellejieli/emotion_text_classifier*, and facial expressions with either *DeepFace* or the HuggingFace model *dima806/facial_emotions_image_detection* [19], [20]. Object perception relies on **YOLOv8n** (Ultralytics) [18], while speech transcription can be performed either locally with **Vosk** or via **Google Speech-to-Text** for higher accuracy [22], [23].

Conversational reasoning is powered by **large language models accessed through the OpenRouter API**, with the main dialogue managed by *deepseek/deepseek-r1-0528-qwen3-8b:free* and a secondary verification layer handled by *Qwen/Qwen3-8B*. Backup models such as GPT-3.5/4 or Llama-3 can be used when needed [8]. For speech synthesis, the system employs **gTTS** for simple outputs and neural TTS models such as *NihalGazi/Text-To-Speech-Unlimited* for more expressive delivery [35]. Finally, orchestration scripts in **Bash** manage the startup and cleanup of modules, ensuring robustness and portability. Since all heavy inference (LLMs and emotion models) is offloaded to external servers, StretchBot can run smoothly on **consumer-grade hardware** without requiring a dedicated GPU, using only a standard laptop, USB webcam, microphone.

## 4.2 Prompt Engineering and Response Format

One of the key challenges in the implementation was to design prompts that make the large language model both adaptive and reliable. The prompt structure had to provide enough context for the model to reason about the user's situation while still producing outputs that could be easily parsed by the robot. To achieve this, the input to the model includes the current exercise, the next planned movement, the user's emotional state, recent dialogue history, and the list of detected objects. This contextualization ensures that the model can adapt its response to both the physical and emotional needs of the user [5], [22], [23].

Several prompting styles were tested during development:

- **Instruction-based prompting**: the model is directly told what role it should play (e.g., "You are StretchBot, a friendly coach…") and what rules it must follow. This provides a clear framework for consistent behavior.

- **Few-shot prompting**: examples of good interactions are included in the prompt to show the model the desired style of reasoning and response. This helps the system imitate successful coaching behaviors.

- **Chain-of-thought prompting**: the model is encouraged to reason step by step before producing its final answer. This makes the decision-making process more transparent and leads to responses that better reflect human commonsense reasoning [5].

The final design combines these approaches. Prompts were written with explicit instructions and a small number of examples, while also requesting the model to "think aloud" before outputting its final decision. To ensure consistency with the robot's control system, responses must begin with predefined action prefixes such as *NEXT_EXERCISE*, *POINT_OBJECT*, or *STOP_ROUTINE*. These rules make it possible for the dialogue manager to directly interpret and execute the LLM's output [31].

Finally, all generated responses are checked by a secondary verification model, which corrects protocol errors such as missing prefixes, repeated offers, or responses that do not account for the user's emotional state. This ensures that the dialogue remains both safe and natural throughout the interaction [33], [35].

## 4.3 Knowledge Representation

For StretchBot to adapt intelligently, it needs not only raw perception but also a structured way of reasoning about what these perceptions mean. To support this, we integrated a lightweight **knowledge graph (KG)** that encodes relations between physical states (e.g., *tired*, *thirsty*, *in pain*) and possible actions or objects that can help (e.g., *drink water*, *eat a banana*, *take a break*). The knowledge graph follows commonsense structures inspired by ConceptNet [5] and related resources, making it easier for the robot to connect user states with relevant suggestions.

For example, the state *tired* is linked to possible remedies such as drinking coffee, eating a banana, or resting. Similarly, the state *thirsty* motivates the action *drink water*, while *in pain* motivates *stop exercise*. Objects such as a *banana* or a *glass of water* are represented with their affordances, such as *provides quick energy* or *satisfies hydration*. These structured relations are retrieved at runtime and embedded into the LLM's prompt, giving the model contextual cues about what is most relevant in the current situation [22], [23].

This representation has two key advantages. First, it ensures **consistency**: the robot will always connect user needs with suitable options, even if the dialogue is phrased differently. Second, it improves **safety and empathy**: by grounding its reasoning in commonsense relations, the system avoids irrelevant or unsafe suggestions and instead offers support that aligns with the user's state [31], [35].

## 4.4 Hardware Setup

The implementation of StretchBot was designed to run on **affordable and accessible hardware**, showing that adaptive human–robot interaction does not require expensive equipment. The perception setup consisted of a **standard USB webcam** for vision and a **basic microphone** for speech input. These low-cost devices were sufficient for object detection, emotion recognition, and dialogue since large-scale inference tasks were delegated to external APIs [22], [23].

For robotic execution, we relied exclusively on the **Lynxmotion SES900 robotic arm**. A **custom URDF model** was developed to simulate the robot in **ROS2 and PyBullet**, ensuring accurate kinematics and smooth trajectory validation before deploying commands on the real robot. **MoveIt** was used for motion planning, and execution was handled by dedicated ROS2 nodes. Safety was ensured through a **watchdog module**, monitoring joint limits and detecting possible collisions [31], [33], [35].

This setup demonstrates that StretchBot can provide reliable adaptive interaction using only **consumer-grade hardware and a lightweight robotic arm**, while leveraging cloud-based AI models for advanced perception and reasoning.

### *4.5 Parameter Settings*

*For our prototype, several implementation parameters were fixed to ensure consistency across trials:*

- ***Exercise monitoring****: In adaptive mode, the hold duration threshold was set to* ***5 seconds****, after which the exercise is considered completed. In the scripted baseline, each exercise was held for a fixed duration of* ***10 seconds****.*

- ***Dialogue adaptation****: Once refused, suggestions (e.g., drinking water) were suppressed for the remainder of the session to avoid repetition.*

- ***Execution timing****: After pointing to an object, the robotic arm maintained its gesture for* ***5 seconds*** *before returning to its home position.*

- ***Latency****: Cloud-based inference introduced delays of up to* ***10–15 seconds****, while scripted execution maintained a constant* ***10-second rhythm****. Local modules such as Vosk STT responded in near real time.*

- ***Audio output****: The voice volume was kept at the default setting, which some participants reported as too low.*

# 5 User Evaluation Design

The evaluation aimed to assess the added value of adaptive behaviour in StretchBot compared to a purely scripted baseline. The focus was on user experience, perceived naturalness of interaction, and overall trust in the robot as a stretching assistant.

**Experimental design**

Two experimental conditions were compared:

- **Scripted (non-adaptive)**: the robot strictly followed a predefined sequence of exercises. It introduced each exercise, waited for a fixed hold duration of 10 seconds, and then moved to the next one without reacting to user state or feedback.

- **Adaptive**: the robot operated with the full reasoning pipeline, integrating perception (emotions, objects), task monitoring, and adaptive decision-making via the LLM. In this condition, the robot could insert pauses, suggest hydration or energy boosts, or skip an exercise if discomfort was expressed. All deviations were collaborative, meaning that the user's confirmation was always required before adapting the routine.

To counterbalance potential order effects, we planned a **within-subject crossover design**:

- Half of the participants first interacted with the adaptive version, followed by the scripted one.

- The other half experienced the reverse order (scripted first, then adaptive).

After each session, participants filled in the same questionnaire to enable paired comparisons between the two conditions.

**Participant selection**

Three participants were recruited for the preliminary evaluation. While this small sample size does not allow for statistical generalization, it provided valuable qualitative insights into how users perceive adaptive vs. non-adaptive robotic guidance. Participants had no prior experience with StretchBot, ensuring that all feedback was based on first-time impressions.

**Evaluation metrics**

User experience was captured through a combination of **quantitative ratings** and **open-ended feedback**:

- **Quantitative ratings**: Six Likert-scale aspects (1 = not at all, 5 = completely) were assessed:
    - clarity of the robot's instructions,
    - comfort during the routine,
    - perceived helpfulness of the robot,
    - adaptability to user state (fatigue, posture, emotion)
    - trust in the robot as a stretching guide,
    - naturalness of the interaction.

- **Open-ended questions**: Participants were invited to elaborate on which version felt better overall, why, and how they perceived the differences between adaptive and scripted behaviour.

In addition to the questionnaire, **direct observation** was used to note participant reactions (hesitations, laughter, verbal comments) during the sessions, which enriched the interpretation of the results.

**Data collection and analysis**

Data was collected via a Google Form immediately after each condition, ensuring that impressions were fresh. The responses were anonymized and stored securely.

Analysis proceeded in two steps:

1. **Quantitative comparison**: Average scores were computed for each question and compared across adaptive vs. scripted conditions.

2. **Qualitative synthesis**: Open-ended responses were thematically coded to identify recurring patterns, such as perceived empathy, trust, or fluidity of the interaction.

The full questionnaire is available at: *[link]*.
Anonymized evaluation results can be accessed here: *[link]*.

# 6. Results and Discussion

This section compares the outcomes of the two evaluated conditions: the scripted (non-adaptive) robot and the adaptive robot. Three participants tested both versions and completed identical questionnaires after each session. The questionnaires included six Likert-scale questions (1–5), followed by open-ended feedback.

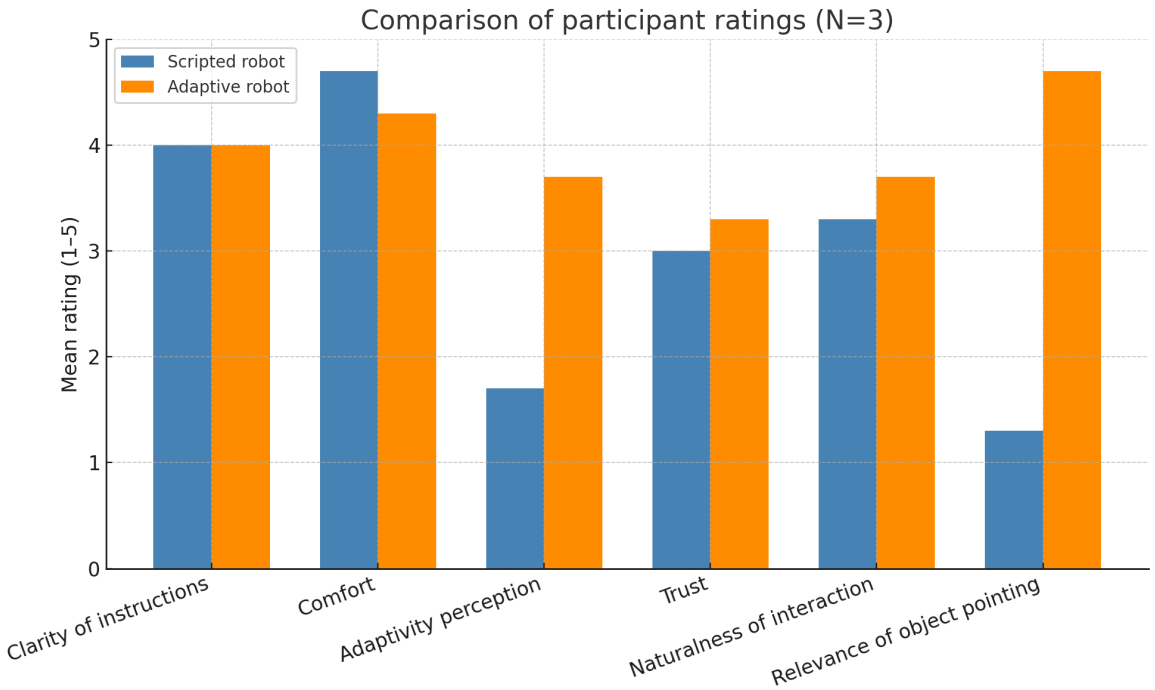


Figure 3 – Mean participant ratings (N=3) across six evaluation criteria, comparing adaptive and non-adaptive robot versions.

## 6.1 Quantitative results

Overall, both versions achieved moderate to positive scores (Figure 3), but notable differences emerged between the two conditions.

- **Clarity of instructions:** Both robots were rated similarly (mean = 4.0 for non-adaptive vs. 4.0 for adaptive). This indicates that participants consistently understood the instructions, regardless of adaptivity.
- **Comfort:** The non-adaptive robot scored slightly higher (mean = 4.7) than the adaptive one (mean = 4.3). Participants may have preferred the predictability of a fixed routine.
- **Adaptivity perception:** As expected, the adaptive version scored higher (mean = 3.7) compared to the scripted one (mean = 1.7), confirming that users noticed the system's ability to react to fatigue, posture, or context.
- **Trust:** Ratings remained modest in both conditions (mean ≈ 3.0–3.3). Trust in robotic guidance appears to be an area requiring improvement.
- **Naturalness of interaction:** Scores were similar (≈3.3–3.7), suggesting that while adaptivity was noticed, it did not strongly enhance perceived naturalness.
- **Relevance of object pointing:** Here, the contrast was significant. The adaptive robot was rated much higher (mean = 4.7) than the scripted version (mean = 1.3). This shows that context-aware pointing (e.g., to coffee when tired) was clearly valued.

Taken together, the non-adaptive robot was often perceived as **more comfortable and efficient**, while the adaptive version was recognized for its **contextual awareness and meaningful gestures**.

## 6.2 Qualitative feedback

Open-ended responses reveal further insights:

- One participant appreciated the **posture detection and corrections** in the adaptive system, describing it as "fun to interact with."
- Another preferred the **scripted version because it was shorter and easier to follow**, highlighting that the adaptive robot sometimes introduced unnecessary delays.
- Suggestions included **quicker response times (GPU acceleration)**, louder voice output, and more explicit explanations of why each stretch is beneficial.

These comments align with the quantitative findings: participants valued the intelligence of the adaptive robot but were discouraged by its slower execution.

## 6.3 Technical and interaction-level observations

During experiments, a key limitation was the **latency introduced by API calls**. The adaptive system sometimes required up to 1**5 seconds of waiting time** before responding, which disrupted the flow of the stretching routine. By contrast, the scripted robot transitioned smoothly every 10 seconds without interruptions.

This difference likely explains why some participants favored the scripted version despite its lack of adaptivity: during physical exercise, users may prioritize **fluidity and efficiency** over contextual personalization.

In addition to latency, most of the difficulties encountered during the study were related to hardware constraints, such as microphone sensitivity, camera quality, and the responsiveness of the robotic arm. These issues sometimes limited the overall smoothness of the interaction.

Another important observation was the initial complexity of interacting with the adaptive system. Even after receiving instructions, the first interaction felt somewhat unintuitive and harder to grasp compared to the scripted robot. This highlights a lack of user-friendliness that will need to be addressed in future iterations, for instance by simplifying the dialogue flow and providing clearer guidance during the first minutes of use.

## 6.4 Interpretation and hypotheses

Two hypotheses emerge from these results:

1. **Preference for uninterrupted exercise:** Participants may prefer a predictable, non-interruptive routine during stretching, where pauses or delays feel like disruptions rather than support.
2. **Latency undermines adaptivity:** The adaptive system's benefits (personalization, object pointing) are overshadowed by technical delays. If the system could respond in near real-time (e.g., <500 ms), perceptions of naturalness and comfort might significantly improve.

## 6.5 Strengths and limitations

- **Strengths of the adaptive system:** meaningful object pointing, recognition of user states, potential for personalization.
- **Strengths of the scripted system:** simplicity, reliability, and smooth execution.
- **Limitations across both:** modest trust scores and the need for clearer explanations of exercise benefits.

# 7. Conclusion and Perspectives

## 7.1 Summary of findings

This project introduced **StretchBot**, an adaptive robotic assistant designed to guide users through a short stretching routine. The system combined multimodal perception (voice, facial expression, text, and object detection) with commonsense reasoning through a knowledge graph and large language models. A comparative user study highlighted clear contrasts: while the scripted robot was perceived as smoother and more predictable, the adaptive version was recognized for its ability to personalize guidance and point to relevant objects in context. However, technical latency and limited user-friendliness reduced the overall experience of adaptivity.

## 7.2 Implications for assistive robotics

These findings emphasize a key challenge in socially assistive robotics: **adaptivity alone is not sufficient if real-time performance and usability are not ensured**. Even when participants valued personalization and contextual support, they preferred predictable interactions when delays or complex dialogues disrupted the exercise flow. For the broader field, this suggests that socially assistive robots must balance **technical robustness** with **human-centered design**, ensuring that adaptive features enhance rather than hinder user experience.

## 7.3 Generalizability to other use cases

Although the study focused on morning stretching, the principles behind StretchBot generalize to other domains:

- **Rehabilitation**: adaptive feedback can help patients perform movements safely while keeping them motivated.

- **Elderly care**: context-aware support (e.g., offering water, suggesting rest) could improve autonomy and wellbeing.

- **Home fitness**: adaptive guidance could make exercise routines more engaging and personalized compared to scripted digital trainers.

In each case, the same integration of multimodal perception, commonsense reasoning, and collaborative interaction protocols could provide value, though adaptations to target populations and contexts would be necessary.

## 7.4 Future work

Several avenues emerge from this study for improving and extending StretchBot:

- **Reducing latency**: current reliance on cloud-based APIs caused delays up to 15 seconds. Deploying models locally on GPU, optimizing the pipeline, or adopting lighter architectures could allow near real-time interaction (<500 ms).

- **Improving user-friendliness**: early interactions showed that participants found the adaptive system harder to use at first. A more intuitive dialogue flow, onboarding instructions, and clearer explanations of exercise benefits could improve accessibility.

- **Enhanced perception**: posture detection and gesture generation would allow more precise feedback, moving beyond verbal guidance toward embodied interaction.

- **Scaling the study**: only three participants tested the system, which limits the conclusions. Larger and more diverse user studies, including elderly participants and rehabilitation contexts, will be essential to confirm findings and refine the design.

- **Expanding functionality**: future iterations could extend beyond stretching to broader wellbeing routines, including strength exercises, relaxation, or rehabilitation therapies, reinforcing the system's value in everyday healthcare and fitness.

*In the future, this line of work could pave the way for adaptive robotic assistants that seamlessly integrate into daily life, acting not only as exercise coaches but also as personalized companions in healthcare, rehabilitation, and home wellbeing.*

# References


[1] D. Feil-Seifer and M. J. Mataric, “Defining socially assistive robotics,” 2005 IEEE 9th International Conference on Rehabilitation Robotics, 2005, pp. 465–468.
[2] A. Tapus, M. J. Mataric, and B. Scassellati, “Socially assistive robotics [Grand Challenges of Robotics],” IEEE Robotics & Automation Magazine, vol. 14, no. 1, pp. 35–42, 2007.
[3] L. Wen, C. H. Lee, and C. H. Hsu, “Design of an intelligent stretching device for physical therapy,” Sensors, vol. 21, no. 16, 2021.
[4] M. Quigley et al., “ROS: an open-source Robot Operating System,” ICRA Workshop on Open Source Software, 2009.
[5] J. Speer et al., “ConceptNet: A practical commonsense reasoning tool-kit,” BT Technology Journal, vol. 22, no. 4, 2004.
[6] F. Tozadore et al., “Stretching robots for stress reduction and focus: a preliminary study,” International Journal of Social Robotics, 2023.
[7] R. Lamsey et al., “Adaptive rehabilitation robot for Parkinson’s patients,” IEEE Transactions on Neural Systems and Rehabilitation Engineering, 2025.
[8] F. Ilievski, P. Szekely, and B. Zhang, “Dimensions of commonsense knowledge,” Proceedings of AAAI, 2021.
[9] M. Ding et al., “Bridging planning and large language models for action reasoning,” NeurIPS, 2023.
[10] R. Kaushik et al., “Adaptive human–robot interaction using multimodal emotion and posture cues,” ACM/IEEE International Conference on Human–Robot Interaction (HRI), 2025.
[11] B. Irfan et al., “Personalized robot interactions to address fatigue and stress,” International Journal of Social Robotics, 2020.
[12] M. Ahn et al., “Do as I can, not as I say: Grounding language in robotic affordances,” arXiv preprint arXiv:2204.01691, 2022.
[13] W. Huang et al., “Inner monologue: Embodied reasoning in language models,” arXiv preprint arXiv:2207.05608, 2022.
[14] C. Zhang et al., “LLMs Can’t Plan: A Survey of Failures in Large Language Model Planning,” ICML, 2024.
[15] AKR3 Research Group, “Hybrid structured knowledge and LLMs for robotic manipulation planning,” AKR3 Workshop on Robotics and AI, 2024.
[16] M. Ciocarlie and K. Hsiao, “Mobile manipulation through an extensible perception, planning, and execution architecture,” Robotics and Autonomous Systems, vol. 62, no. 4, pp. 507–521, 2014.
[17] N. Kohl and P. Stone, “Machine learning for fast quadrupedal locomotion,” AAAI, 2004.
[18] G. Jocher et al., “YOLO by Ultralytics,” GitHub repository, 2023. Available: https://github.com/ultralytics/ultralytics
[19] T. Wolf et al., “Transformers: State-of-the-Art Natural Language Processing,” EMNLP, 2020.
[20] S. Serengil and A. Ozpinar, “LightFace: A hybrid deep face recognition framework,” 2020 Innovations in Intelligent Systems and Applications Conference (ASYU), IEEE, 2020.
[21] Z. Cao et al., “OpenPose: Realtime multi-person 2D pose estimation using Part Affinity Fields,” CVPR, 2017.
[22] A. Kaldi team, “Vosk Speech Recognition Toolkit,” 2021. Available: https://alphacephei.com/vosk
[23] Google Cloud, “Speech-to-Text API Documentation,” Google Cloud Platform, 2023.
[24] Y. Bengio et al., “Curriculum learning,” ICML, 2009.
[25] S. Russell and P. Norvig, Artificial Intelligence: A Modern Approach, 4th ed., Pearson, 2020.
[26] A. Hogan et al., “Knowledge Graphs,” ACM Computing Surveys, vol. 54, no. 4, pp. 1–37, 2021.
[27] J. Park et al., “Structured prompting for large language models,” arXiv preprint arXiv:2305.00050, 2023.
[28] S. Lin et al., “Teaching language models to self-check,” NeurIPS, 2022.
[29] D. Amodei et al., “Learning from human feedback for safe reinforcement learning,” arXiv preprint arXiv:1606.06565, 2016.

[30] S. Tellex et al., “Approaching the symbol grounding problem with probabilistic graphical models,” AI Magazine, vol. 34, no. 4, pp. 64–76, 2013.
[31] I. A. Şucan and S. Chitta, “MoveIt! motion planning framework,” IEEE Robotics & Automation Magazine, vol. 19, no. 1, pp. 18–19, 2012.
[32] C. de Boor, A Practical Guide to Splines, Springer, 1978.
[33] Lynxmotion, “LSS 6 DOF Robotic Arm (SES V2),” Lynxmotion documentation, 2021.
[34] A. Kendon, Gesture: Visible Action as Utterance, Cambridge University Press, 2004.
[35] Y. Wang et al., “Tacotron: Towards end-to-end speech synthesis,” Interspeech, 2017.
[36] J. Nakanishi et al., “Operational space control: A theoretical and empirical comparison,” International Journal of Robotics Research, vol. 27, no. 6, pp. 737–757, 2008.
[37] OptiTrack, “Optical motion capture system documentation,” NaturalPoint Inc., 2022.
[38] A. Alami et al., “Safe and dependable physical human–robot interaction in anthropic domains: State of the art and challenges,” IEEE Transactions on Robotics, vol. 22, no. 5, pp. 871–885, 2006.
[39] J. Hoffmann, “Everything You Always Wanted to Know About Planning (But Were Afraid to Ask),” AI Communications, vol. 33, no. 1, pp. 1–32, 2020.
[40] S. Srivastava et al., “Combined task and motion planning through an extensible planner-independent interface layer,” IEEE ICRA, 2014.
[41] T. Lozano-Pérez and L. P. Kaelbling, “A constraint-based method for solving sequential manipulation planning problems,” IEEE/RSJ IROS, 2014.
[42] A. Tapus and M. J. Mataric, “User personality matching with a socially assistive robot for post-stroke rehabilitation,” Journal of Human–Robot Interaction, vol. 1, no. 2, pp. 158–180, 2012.
[43] C. Breazeal, Designing Sociable Robots, MIT Press, 2002.
[44] D. Kortenkamp and T. Weymouth, “Topological mapping for mobile robots using a semantic hierarchy of spatial representations,” Proceedings of the 1994 AAAI, pp. 979–984.
[45] M. Beetz et al., “Plan-based control of robotic agents,” The International Journal of Robotics Research, vol. 18, no. 6, pp. 733–759, 1999.
[46] K. Dautenhahn, “Socially intelligent robots: dimensions of human–robot interaction,” Philosophical Transactions of the Royal Society B: Biological Sciences, vol. 362, no. 1480, pp. 679–704, 2007.